

第三章（二）浮点运算与加法器

1. 浮点加减中的对阶是()

- A. 将较大的一个阶码调整到与较小的一个阶码相同
- B. 将加数的阶码调整到与被加数的阶码相同
- C. 将被加数的阶码调整到与加数的阶码相同
- D. 将较小的一个阶码调整到与较大的一个阶码相同

正确答案：D

解析：浮点数加减运算第一步为**对阶**，固定规则是**小阶向大阶看齐**。

若把大阶改成小阶，大数尾数需要左移，会丢失高位有效数字，精度大幅受损；

将小阶提升至和大阶相等，小数尾数仅右移，只会丢失低位，精度损失最小。

2. 浮点数运算的溢出判断，取决于()

- A. 尾数是否下溢
- B. 尾数是否上溢
- C. 阶码是否下溢
- D. 阶码是否上溢

正确答案：D

解析：浮点数的表示范围由阶码决定，精度由尾数决定：

1) **阶码上溢**：阶码超过硬件能存储的最大值，运算结果超出可表示数值范围，判定为溢出；

2) **阶码下溢**：阶码小于最小值，数值无限趋近 0，机器直接置零，不属于溢出；

3) **尾数上下溢**仅影响精度，可通过规格化移位修正，不会产生溢出。

3. 下列关于整数乘法运算的叙述中，错误的是()

- A. 用阵列乘法器实现乘运算可以在一个时钟周期完成
- B. 用 ALU 和移位器实现的乘运算无法在一个时钟周期内完成

- C. 两个变量的乘运算无法编译为移位及加法等指令的循环实现
- D. 变量与常数的乘运算可编译优化为若干位移位及加减运算指令

正确答案：C

解析：

A：阵列乘法器是并行组合逻辑，一次性生成所有部分积并求和，单周期完成乘法，正确；

B：普通 ALU + 移位器为串行乘法，每轮仅完成一次移位 + 加法，多周期执行，正确；

C（错误选项）：任意两个变量相乘，都能拆解为循环移位、加法指令实现（补码一位乘、原码一位乘均为原理）；

D：常数乘法可优化，例如 $x * 5 = x \ll 2 + x$ ，编译器会替换乘法指令，提升效率，正确。

4. 在浮点数原码运算时，判定结果为规格化数的条件是（ ）

- A. 阶的符号位与尾数的符号位不同
- B. 尾数的符号位与最高数值位不同
- C. 尾数的符号位与最高数值位相同
- D. 尾数的最高数值位为 1

正确答案：D

解析：本题限定**原码尾数**，区分原码、补码规格化规则：

二进制原码规格化要求尾数绝对值 ≥ 0.1 ，即尾数最高有效数值位必须为 1；

补充区分：补码规格化规则是「尾数符号位与最高数值位不同」，不要混淆两者。

5. 加法器采用先行进位的目的是（ ）

- A. 增强加法器结构
- B. 优化加法器的结构
- C. 节省器材

D. 加速传递进位信号

正确答案：D

解析：串行进位加法器：进位从最低位逐位传递到最高位，位数越多延迟越大，运算速度慢

先行进位（超前进位 CLA）：并行计算每一位进位输出，无需等待低位进位传递，核心目的是加快进位信号传输；

A、B 只是电路带来的附加变化，不是设计目的；先行进位需要额外逻辑门，无法节省硬件，C 错误。

6. 用补码一位乘计算 $x*y$ （写出具体的运算过程）， $x=0.10110$ ， $y=-0.00011$

正确答案：1.1110111110

解析：Booth 运算法则

求补码：为正数，原码=补码， $[x]_{补} = 00.10110$ ， $[-x]_{补} = 11.01010$

Y 为负数，原码=1.00011，符号不变，数值位取反加 1 得补码 $[y]_{补} = 11.11101$

y-1 的首位减去乘数低位的末尾，为负数则加 $[-x]_{补}$ ，为正数则加 $[x]_{补}$ ，为 0 则加 0.00000

部分积	乘数低位	y-1
00.00000	11101	0
+ 11.01010		
11.01010		
→11.10101	01110	1
+ 00.10110		
00.01011		
→00.00101	10111	01
+ 11.01010		
11.01111		
→11.10111	11011	101
+ 00.00000		
11.10111		
→11.11011	11101	1101
+ 00.00000		
11.11011		
→11.11101	11110	11101

因此 $[x * y]_{补} = 1.1110111110$